

SHARIF  
UNIVERSITY OF TECHNOLOGY



---

# Computer Structure

*Pre-Lab Report 3*

---

Aria Fateh Kerdari  
403110382

June 13, 2026

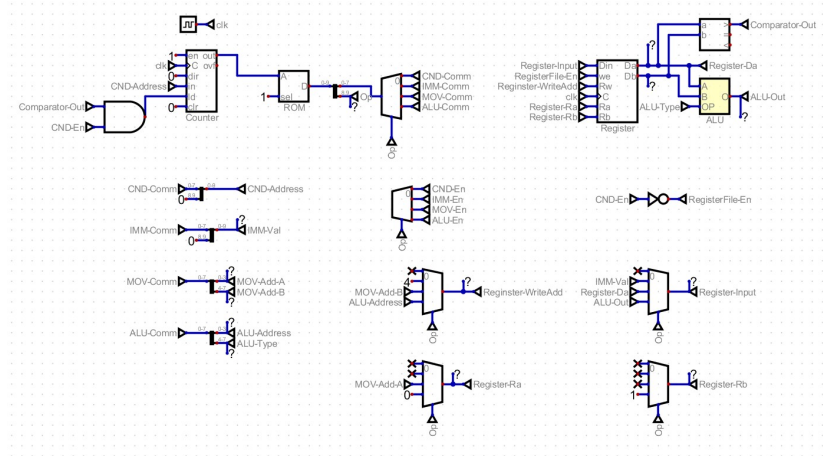


Figure 1: Simplified microprocessor.

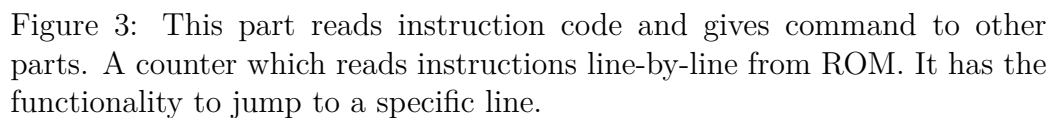
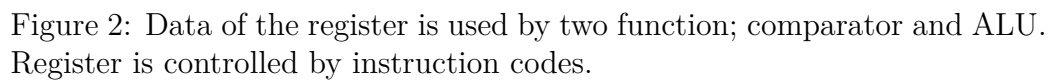
## 1 Introduction

This project focuses on the design and programming of a simplified custom microprocessor architecture. The processor is designed to execute specific tasks using a minimized Instruction Set Architecture (ISA), optimized for hardware efficiency and low-level control. Every instruction in this architecture is exactly 10 bits wide, divided into a 2-bit Opcode and an 8-bit Operand.

The primary objective of this study was to implement a persistent control loop for a battery management system. This system monitors battery levels, handles automatic discharging from 80% to 20%, and initiates a charging cycle back up to 90%. To bridge the gap between high-level logic and hardware-level binary, a dedicated Assembler and a Hex-Converter were developed in Python to automate the generation of ROM-ready code.

## 2 RTL Realization

The implementation is shown in figures below.



## 3 Instruction Set Architecture (ISA)

The processor utilizes a 10-bit instruction format:

- **Opcode [2 bits]:** Defines the operation type.
- **Value/Address [8 bits]:** Provides immediate data or memory addresses.

### 3.1 Operations

1. **CND (00):** Conditional jump. If `R0` > `R1` it jumps to a specific address.
2. **IMM (01):** Loads an immediate 8-bit value into `R4` internal register.
3. **MOV (10):** Transfers data between registers to target registers.
4. **ALU (11):** Performs arithmetic operations. `ADD` and `SUB`.

### 3.2 The Assembler

The Assembler script translates human-readable mnemonic instructions (e.g., `IMM 80`) into their corresponding 10-bit binary strings. It parses the source code line-by-line, ignores comments, and handles register mapping. The output is a text file where each line represents a 10-bit instruction, making it compatible with hardware simulation environments or ROM initialization files.

### 3.3 Binary-to-Hex Converter

To support various hardware memory formats (such as Intel HEX or raw ROM files), a conversion tool was created. This script reads the 10-bit binary strings generated by the assembler and packs them into 16-bit words (2 bytes), which are then converted into Hexadecimal format. This ensures that the code can be easily flashed into physical ROM modules.

## 4 System Implementation: Battery Control Loop

The core application demonstrates a state-machine-like behavior for battery management. Since the ISA lacks an unconditional jump ( `JMP` ), a "forced condition" technique was utilized. By subtracting a large value from a smaller one, we manually trigger the negative flag to force the `CND` instruction to act as a jump.

### 4.1 Program Logic

The program is divided into four main sections:

- **Initialization:** Sets the battery level (`R0`) to 80 and defines constants for 20% and 90% thresholds.
- **Discharge Loop:** Subtracts 4 units per step. After each step, it compares the battery to the 20% threshold. If reached, it branches to the Charge Loop.
- **Charge Loop:** Increases the battery by 6 units per step until it reaches 90%, then jumps back to the discharge phase.
- **Unconditional Jump Trick:** Uses `IMM 100, MOV R1,` and `SUB R1` to force a jump back to the start of a loop.